

GRUPO I

1) static String arcoIrisAleatorio(String[] cores, int n)

O que faz: escolhe **n** cores ao acaso do vetor **cores** e junta-as numa **String** separada por espaços.

Passos

1. Criar **Random** **r**
2. Criar **StringBuilder** **sb**
3. Repetir **n** vezes:
 - o escolher índice aleatório **k = r.nextInt(cores.length)**
 - o **sb.append(cores[k])**
 - o se não for a última, **sb.append(" ")**
4. devolver **sb.toString()**

```
static String arcoIrisAleatorio(String[] cores, int n) {
    Random r = new Random();
    StringBuilder sb = new StringBuilder();

    for (int i = 0; i < n; i++) {
        int k = r.nextInt(cores.length);
        sb.append(cores[k]);
        if (i < n - 1) sb.append(" ");
    }
    return sb.toString();
}
```

3) Classes Grupo1Exerc3 e SubGrupo1Exerc3

Regras rápidas do código

- Em **Grupo1Exerc3**:
 - o **x** guarda o número
 - o **sb** começa vazio
 - o **addText(v, s)** só acrescenta **s** se **isValid(v)** for **true**
 - o **incrementX(i, v)** soma **i** a **x** se **i < v**
 - Em **SubGrupo1Exerc3**:
 - o **t** é a string extra
 - o **isValid(i)** é: **i > getX()**
 - o **contents()** devolve **super.contents() + t**
- Exame1-19_20

3(a)

- **"PC0".length() = 3** → **x = 3**
- **sb = ""**
- **t = "PC0"**
- imprime **getX()** → **3**
- i) **Atributos:** **x=3, sb="", t="PC0"**
- ii) **Ecrã:** **3**

3(b)

- começa: **x=2, sb="", t="P"**
- **addText(5, "C")**:
 - o **isValid(5)** → **5 > x(2)** → **true**
 - o então **sb = "C"**
- **contents()** → **super.contents()** é **"C"** e depois + **t ("P")** → **"CP"**
- i) **Atributos:** **x=2, sb="C", t="P"**
- ii) **Ecrã:** **CP**

3(c)

- **"01ah".length() = 4** → **x=4, sb="", t="01ah"**
- **incrementX(2, 6)**:

- $2 < 6 \rightarrow \text{sim} \rightarrow x = 4 + 2 = 6$
- `isValid(3) → 3 > 6 → false`
- então imprime `getX() → 6`
 - i) Atributos: `x=6, sb="", t="Olá"`
 - ii) Ecrã: 6

GRUPO II (classes de apostas)

II.1) AbstractBetableEvent

O que esta classe faz: guarda a parte comum:

- `minBet, maxBet`
- lista de `Better` registados
- lógica do “próximo better” (`nextBetter()`)

```
public abstract class AbstractBetableEvent implements BetableEvent {
    private int minBet, maxBet;
    private List<Better> betters = new ArrayList<>();
    private int next = 0;

    public AbstractBetableEvent(int minBet, int maxBet) {
        this.minBet = minBet;
        this.maxBet = maxBet;
    }

    @Override public int minBet() { return minBet; }
    @Override public int maxBet() { return maxBet; }

    @Override
    public void registerBetter(String name) {
        betters.add(new Better(name));
    }

    @Override
    public int numberBetters() {
        return betters.size();
    }

    @Override
    public Better nextBetter() {
        Better b = betters.get(next);
        next = (next + 1) % betters.size();
        return b;
    }
}
```

II.2) OneByOneBetableEvent

- O evento tem um **valor fixo secreto** dentro [`minBet, maxBet`]
 - Cada `makeBet()` só deixa **um** better apostar (o `nextBetter()`)
 - Fecha quando alguém acerta; esse é o único vencedor
- Exame1-19_20

```
public class OneByOneBetableEvent extends AbstractBetableEvent {
    private int value;
    private boolean closed = false;
    private String winner = null;

    public OneByOneBetableEvent(int minBet, int maxBet) {
        super(minBet, maxBet);
        Random r = new Random();
        this.value = r.nextInt(maxBet - minBet + 1) + minBet;
    }
}
```

```

@Override
public void makeBet() {
    if (closed()) return;

    Better b = nextBetter();
    int bet = b.makeBet(minBet(), maxBet());
    if (bet == value) {
        closed = true;
        winner = b.name();
    }
}

@Override
public boolean closed() {
    return closed;
}

@Override
public Iterable<String> winnerBetters() {
    if (!closed || winner == null) return List.of();
    return List.of(winner);
}
}

```

II.3 Igualdade (**equals**) e **hashCode**

Quando implementas igualdade, a regra base é:

- Se dois objetos são “iguais” com **equals**, então **têm de ter o mesmo hashCode**.

Uma solução simples e segura aqui é:

- Em **AbstractBetableEvent**: comparar **minBet** e **maxBet** e a classe
- Em **OneByOneBetableEvent**: além disso comparar o **value** (faz sentido porque define o evento)